

Problem Set 9

Due Date November 22
Name **Hyma Jujuru**
Student ID **110743269**
Collaborators **Evan Feng**

Contents

Instructions	1
Honor Code (Make Sure to Virtually Sign)	2
1 Hash Tables and List Doubling	3
1.1 Problem 1 [15 pts]	3
1.2 Problem 1 [20 pts]	4
1.3 Problem 3 [15 pts]	5
1.4 Problem 4 [50 pts]	6

Instructions

- The solutions **should be typed**, using proper mathematical notation. We cannot accept hand-written solutions. Here's a short intro to $\text{\LaTeX}$.
- You should submit your work through the **class Gradescope page** only (linked from Canvas). Please submit one PDF file, compiled using this $\text{\LaTeX}$ template.
- You may not need a full page for your solutions; pagebreaks are there to help Gradescope automatically find where each problem is. Even if you do not attempt every problem, please submit this document with no fewer pages than the blank template (or Gradescope has issues with it).
- You are welcome and encouraged to collaborate with your classmates, as well as consult outside resources. You must **cite your sources in this document**. **Copying from any source is an Honor Code violation. Furthermore, all submissions must be in your own words and reflect your understanding of the material.** If there is any confusion about this policy, it is your responsibility to clarify before the due date.
- Posting to **any** service including, but not limited to Chegg, Reddit, StackExchange, etc., for help on an assignment is a violation of the Honor Code.
- You **must** virtually sign the Honor Code (see Section). Failure to do so will result in your assignment not being graded.

Honor Code (Make Sure to Virtually Sign)]

- My submission is in my own words and reflects my understanding of the material.
- Any collaborations and external sources have been clearly cited in this document.
- I have not posted to external services including, but not limited to Chegg, Reddit, StackExchange, etc.
- I have neither copied nor provided others solutions they can copy.

Agreed (signature here). I agree to the above, Hyma Jujjuru.

□

1 Hash Tables and List Doubling

1.1 Problem 1 [15 pts]

Problem 1. Hash tables and balanced binary trees can be both be used to implement a dictionary data structure, which supports insertion, deletion, and lookup operations. In balanced binary trees containing n elements, the runtime of all operations is $\Theta(\log n)$.

For each of the following three scenarios, compare the average-case performance of a dictionary implemented with a hash table (which resolves collisions with chaining using doubly-linked lists) to a dictionary implemented with a balanced binary tree.

- (a) A hash table with hash function $h_1(x) = 1$ for all keys x .

Answer. When $h_1(x) = 1$ for all keys x

	Hash Dictionary	Balanced BST Dictionary
Insertion	$O(1)$	$\Theta(\log n)$
Deletion	$O(n)$	$\Theta(\log n)$
Lookup	$O(n)$	$\Theta(\log n)$

□

- (b) A hash table with a hash function h_2 that satisfies the Simple Uniform Hashing Assumption, and where the number m of buckets is $\Theta(n)$.

Answer. number m buckets is $\Theta(n)$

	Hash Dictionary	Balanced BST Dictionary
Insertion	$O(1)$	$\Theta(\log n)$
Deletion	$O(1)$	$\Theta(\log n)$
Lookup	$O(1)$	$\Theta(\log n)$

□

- (c) A hash table with a hash function h_3 that satisfies the Simple Uniform Hashing Assumption, and where the number m of buckets is $\Theta(n^{3/4})$.

Answer. number m buckets is $\Theta(n^{3/4})$

	Hash Dictionary	Balanced BST Dictionary
Insertion	$O(n^{1/4})$	$\Theta(\log n)$
Deletion	$O(n^{1/4})$	$\Theta(\log n)$
Lookup	$O(n^{1/4})$	$\Theta(\log n)$

□

1.2 Problem 1 [20 pts]

Problem 2. Given the balanced binary trees T_1 and T_2 , which contain m and n elements respectively, we want to determine whether they have some particular key in common. Assume an adversarial sequence that placed the m and n items into the two trees.

- (a) Suppose our algorithm traverses each node of T_1 using an in-order traversal and checks if the key corresponding to the current node traversed exists in T_2 . Express the asymptotic running time of this procedure, in terms of m and n .

Answer. for Part a)

In-Order Traversal of T_1 : an in-order traversal of a binary tree visits each node only once so its runs $O(m)$ times.

Checking if corresponding key exists in T_2 : to check if a key from T_1 exists in T_2 , we perform a search operation and searching a balanced binary tree takes $O(\log n)$ runtime.

Combining these two gives us: $O(m) + O(m \cdot \log n) = O(m \cdot \log n)$

Asymptotic Running Time: $O(m \cdot \log n)$

□

- (b) Now suppose our algorithm first allocates a new hash table H_1 of size m (assume H_1 uses a uniform hash function) and then inserts every key in T_1 into H_1 during a traversal of T_1 . Then, we traverse the tree T_2 and search for whether the key of each node traversed already exists in H_1 . Give the asymptotic running time of this algorithm in the average case. Justify your answer.

Answer. for Part b)

Building Hash Table H_1 : When inserting every key in T_1 into H_1 , we traverse T_1 . The traversal takes $O(m)$ time and insertion takes $O(1)$ on average due to the uniform hash function. So, the total time for building H_1 is $O(m)$.

Traverse Tree T_2 and search if key exists in H_1 : Traversal of T_2 takes $O(n)$ time and searching in a hash table which uses uniform hash function is $O(1)$ on average. So, the total running time for traversing Tree T_2 and searching if a key from T_2 exists in hash table H_1 is $O(n) + O(n) = O(n)$

Combining the two parts gives us: $O(m) + O(n) = O(m + n)$

Asymptotic Runtime: $O(m + n)$

□

1.3 Problem 3 [15 pts]

Problem 3. Consider a hash table A that holds data from a fixed, finite universe U . If the hash table starts empty with $b = 10$ buckets and doubles its number of buckets whenever the load factor exceeds $\frac{1}{2}$, what is the load factor after adding $n = 36$ elements to the hash table?

Proof. Answer.

Load Factor $\lambda = \frac{n}{b}$

The hash table starts with 10 buckets. The load factor exceeds $\frac{n}{b} > \frac{1}{2}$. So, $n > \frac{b}{2} = \frac{10}{2} = 5$. Therefore, when $n = 6$, the load factor will exceed $\frac{1}{2}$ and the number of buckets will be 20.

When $n = 20$, The load factor $\lambda = \frac{n}{b} = \frac{20}{20} = 1$ which is greater than $\frac{1}{2}$ so the number of buckets doubles to 40.

With $n = 21$, load factor $\lambda = \frac{n}{b} = \frac{21}{40} = 0.525$ which is greater than $\frac{1}{2}$ so the number of buckets doubles to 80.

For $n = 36$, the load factor $\lambda = \frac{36}{80} = 0.45$

Load Factor After Adding 36 elements to the Hash Table: 0.45 or 45%

□

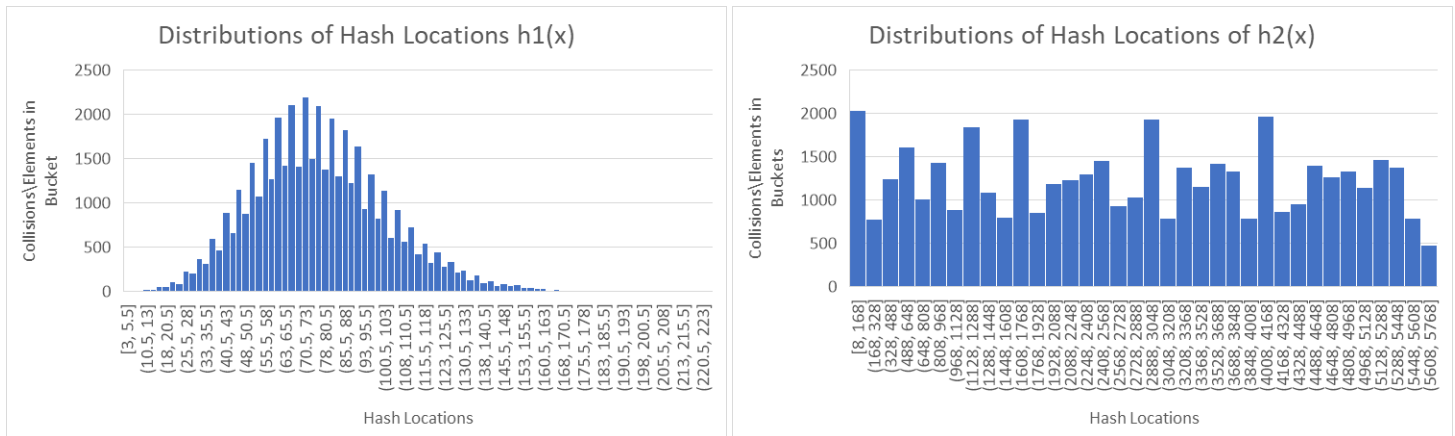
1.4 Problem 4 [50 pts]

Problem 4. A good hash function $h(x)$ behaves in practice very close to the uniform hashing assumption analyzed in class, but is a deterministic function. That is, $h(x) = k$ each time x is used as an argument to $h()$. Designing good hash functions is hard, and a bad hash function can cause a hash table to quickly exit the sparse loading regime by overloading some buckets and underloading others. Good hash functions often rely on beautiful and complicated insights from number theory, and have deep connections to pseudorandom number generators and cryptographic functions. In practice, most hash functions are moderate to poor approximations of uniform hashing.

Consider the following two hash functions. Let U be the universe of strings composed of the characters from the alphabet $\Sigma = [A, \dots, Z]$, and let the function $f(x_i)$ return the index of a letter $x_i \in \Sigma$, e.g., $f(A) = 1$ and $f(Z) = 26$. Finally, for an m -character string $x \in \Sigma^m$, define $h1(x) = ([\sum_{i=1}^m f(x_i)] \bmod \ell)$, where ℓ is the number of buckets in the hash table. For the other hash function, let the function $f2(x_i, a_i)$ return $f(x_i) * a_i$, where a_i is a uniform random integer, $a_i \in [0, \dots, \ell-1]$, and define $h2(x) = ([\sum_{i=1}^m f2(x_i, a_i)] \bmod \ell)$. That is, the first hash function sums up the index values of the characters of a string x and maps that value onto one of the ℓ buckets, and the second hash function is a universal hash function.

- There is a txt file that contains US Census derived last names:
Using these names as input strings, first choose a uniformly random 50% of these name strings and then hash them using $h1(x)$ and $h2(x)$.
- Produce a histogram showing the corresponding distribution of hash locations for each hash function when $\ell = 5701$. Label the axes of your figure. Brief description what the figure shows about $h1(x)$ and $h2(x)$; justify your results in terms of the behavior of $h(x)$.

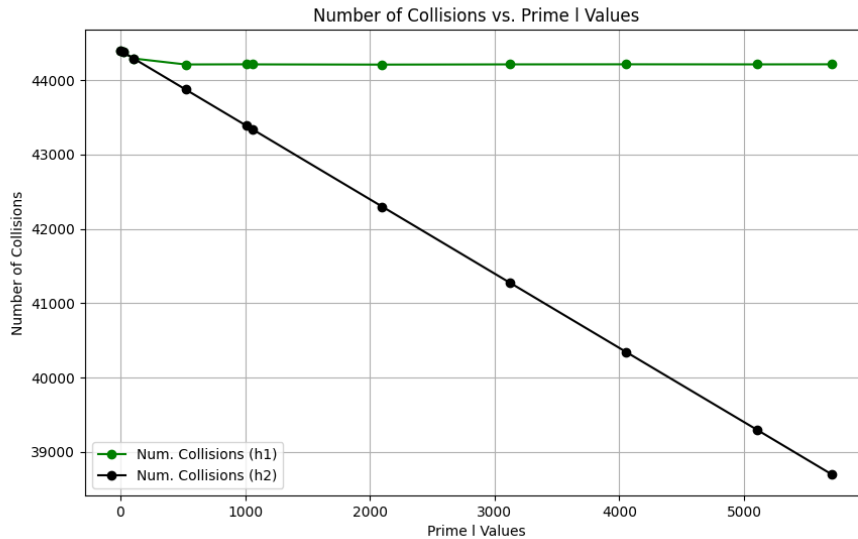
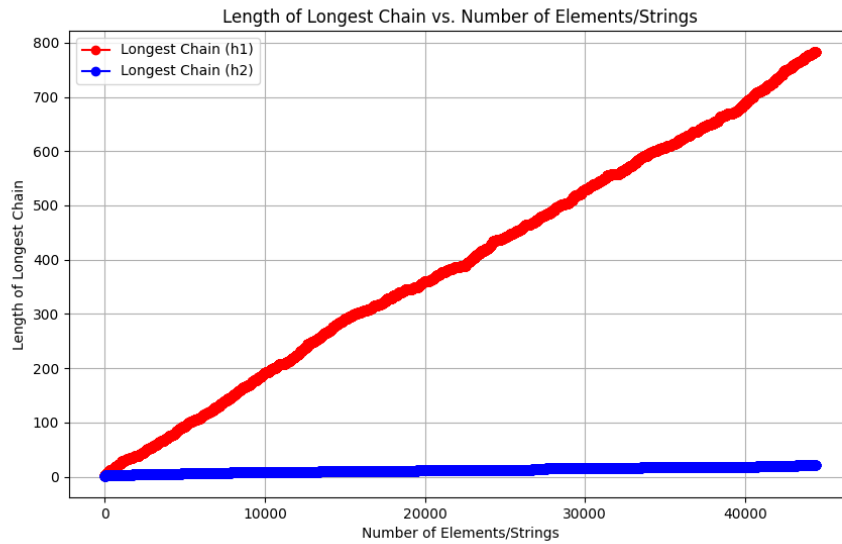
Hint: the raw file includes information other than the name strings, which will need to be removed; and, think about how you can count hash locations without building or using a real hash table.



The figure of $h1(x)$ is a slightly skewed bell curve which shows that some buckets have more elements than others because of how some characters have more frequency in the English alphabet as compared to others. The figure of $h2(x)$ is more equally distributed (less curvy) as it uses a universal hash function which means elements are assigned to buckets more fairly or evenly.

- (c) Enumerate at least 4 reasons why $h_1(x)$ is a bad hash function relative to the ideal behavior of uniform hashing.
- 1) **Lack of Randomness/Frequency of Letters:** some letters appear more in the English language so the distribution is not equal so some keys have more collisions than others
 - 2) **Word Size:** Words differ in sizes so the strings we are adding to the hash table could be very large. So, the number of buckets given the size could create more collisions. On the other hand, smaller words can lead to clustering.
 - 3) **Same Sums:** Strings like “HI” and “IH” have the same sums so they will produce the same hash value, which will cause collisions of combinations of these characters. A good hash function should distribute based on all aspects of input to reduce collisions or clustering
 - 4) **Hash Table Size:** Depending on the text given and the number of words in U , a large table (ℓ is big) will have fewer collisions but take up more memory. $h_1(x)$ is not uniform so some buckets will have a significantly longer collision chain than others. With a large ℓ , $h_1(x)$ has many buckets to fill as in part b where there are 5701 buckets, yet $h_1(x)$ used less than 200 buckets which shows how $h_1(x)$ has a lot more clustering and collisions $h_2(x)$ is uniform so the collisions are more evenly spread.

- (d) Produce a plot showing (i) the length of the longest chain (were we to use chaining for resolving collisions) as a function of the number n of these strings that we hash into a table with $\ell = 5701$ buckets. Produce another plot showing the number of collisions as a function of ℓ . Choose prime numbers for ℓ and comment on how collisions decrease as ℓ increases.



The hash function $h1(x)$ will work the same regardless of the number of buckets after a certain point because the most frequent character will always be the same. Yet, in uniform distribution as in has function $h2(x)$, the elements are split more evenly across the buckets which means there will be less collisions as ℓ increases as there will be more buckets to fill the elements into.